A-maz-ing

Requisitos Generales:

1. Uso de try, except y with cuando es necesarios
2. Debe pasar correctamente mypy y flake8
3. Todas las funciones deben tener docstrings siguiendo *Google or NumPy style*

Requisitos del Makefile:

1. *install:* Para instalar todas las dependencias necesarias
2. *run:* Para ejecutar el script principal del proyecto
3. *debug:* Ejecutar el script principal usando el built-in debugger de Python
4. *clean*: Remueve ficheros temporales (**_pycache**, .mypy*cache, etc)*
5. *lint:* Ejecuta flake8 y mypy con las banderas: --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs
6. *lint-strict*: *flake8* y *mypy* con --strict (flag)
7. **Test**: Crear programas de test solo para la defensa usando frameworks como pytest o unittest

Concepto:

- el programa sera un generador de laberintos que toma un archivo de configuracion y genera un laberinto casi perfecto(con apenas un camino entre la entrada y la salida) y lo escriba en un archivo, representando las paredes de las celdas en hexadecimal, como el ejemplo:

  0x0 = 0000 en binario = SIN paredes (totalmente abierto)
  0x1 = 0001 en binario = pared NORTH
  0x2 = 0010 en binario = pared EAST
  0x3 = 0011 en binario = pared NORTH + EAST
  0x4 = 0100 en binario = pared SOUTH
  0x5 = 0101 en binario = pared NORTH + SOUTH
  0x6 = 0110 en binario = pared EAST + SOUTH
  0x7 = 0111 en binario = pared NORTH + EAST + SOUTH
  0x8 = 1000 en binario = pared WEST
  0x9 = 1001 en binario = pared NORTH + WEST
  0xA = 1010 en binario = pared EAST + WEST
  0xB = 1011 en binario = pared NORTH + EAST + WEST

0xC = 1100 en binario = pared SOUTH + WEST
0xD = 1101 en binario = pared NORTH + SOUTH + WEST
0xE = 1110 en binario = pared EAST + SOUTH + WEST
0xF = 1111 en binario = TODAS las paredes (totalmente cerrado)

- ```
  ![[compass-ross-cardinal-points.png|238]]
  ```

Uso:

- El archivo se debe correr con: a_maze_ing.py (nombre del archivo) y config.txt(argumento).

Errores:

- Se pueden lanzar errores como configuracion invalida, archivo no encontrado, mala sintaxis o parametro de laberinto imposible.
- No puede Creashear inesperadamente y debe mostrar mensajes de error claro.

Configuracion del archivo:

- El archivo de contener un par 'KEY=VALUE' por linea.
- Las lineas que comienzan con '#' son comentarios y deben ser ignoradas.
- **Keys obligatorias:**

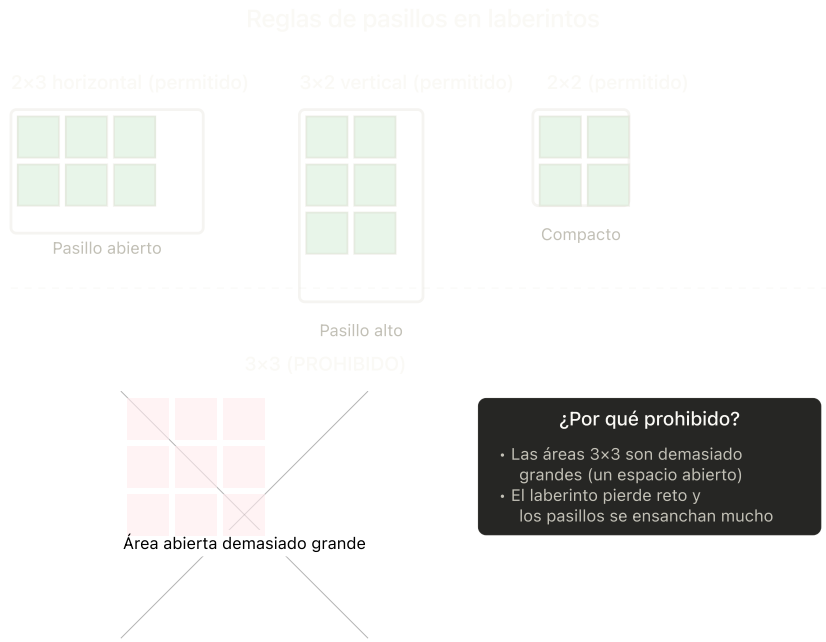
Key	Description	Example
WIDTH	Maze width (number of cells)	WIDTH=20
HEIGHT	Maze height	HEIGHT=15
ENTRY	Entry coordinates (x,y)	ENTRY=0,0
EXIT	Exit coordinates (x,y)	EXIT=19,14
OUTPUT_FILE	Output filename	OUTPUT_FILE=maze.txt
PERFECT	Is the maze perfect?	PERFECT=True

  - Debemos agregar adicional keys caso sea necesario, ejemplo: seed, algorithm, display mode, etc.
  - Se debe enviar junto con el proyecto un un archivo de configuracion default.

Requisitos del laberinto:

- Se debe generar de manera aleatoria pero debe poder ser reproducible mediante una seed.
- Cada celda dentro del laberinto tiene entre 0 y 4 muros, uno a cada punto cardinal.
- El laberinto de ser valido y para serlo:
  - La entrada y la salida existen y se encuentran dentro de los limites del laberinto.
  ```

- La estructura debe asegurar total conectividad entre celda, no celdas aisladas(excepto por el patron 42).
- Las celdas de entrada y salida son especificas por lo que deben tener muros en los bordes externos.
- Debe haber coherencia entre los muros de las celdas, es decir que si tenemos dos celdas contiguas y la de la izquierda(viendolas de frente) tiene muro derecho, la de la derecha, debe tener muro izquierdo.
- El laberinto no puede tener corredores con mas de dos celdas de ancho. Solo se pueden tener areas: 2x3 o 3x2, pero nunca 3x3:



- En la representacion visual el laberinto debe tener un '42' visiblemente dibujado por varias celdas totalmente cerradas. El subject en la pagina 8 dice que en caso de que las dimensiones sean muy pequenas, se puede omitir el patron pero tambien dice que se debe imprimir un error en la consola. No deja claro si se debe continuar ejecutando el programa o no.
- Si la FLAG=PERFECT es activada el laberinto debera contener apenas un camino entre la entrada y la salida.

Formato del archivo de salida:

- El laberinto debe ser descrito en hexadecimal, en cada linea se describe cada celda usando el formato explicado arriba en el apartado de ***Concepto***.
- Después de describir todo el laberinto se debe dejar una linea en blanco y luego escribir las coordenadas (x,y) de la celda de entrada y salida, ejemplo: 1,1. Que seria la celda superior izquierda (la esquina izquierda).

- Luego como ultima linea se debe mostrar el camino mas corto entre el punto de entrada y salida usando la primera letra del punto cardinal por el que se avanzo para salir de cada celda desde el punto de entrada hasta el punto de salida. Ejemplo: Si de la primera celda salimos por el muro inferior la primera letra sera *S*, en la segunda celda salimos por el muro de la derecha, entonces agregamos la *E*, por ahora la linea iria: *SE*. Asi sucesivamente.
- Todas las lineas deben acabar con "\n".
- Este archivo sera testado junto con el archivo de configuración por moulinette para verificar que el output tiene data coherente. Tenemos un script para testar esto.
- Ejmplo del archivo:

```
bash-3.2$ cat output_maze.txt
9515391539551795151151153
EBABAE812853C1412BA812812
96A8416A84545412AC4282C2A
C3A83816A9395384453A82D02
96842A852AC07AAD13A8283C2
C1296C43AAB83AA92AA8686BA
92E853968428444682AC12902
AC3814452FA83FFF82C52C42A
85684117AFC6857FAC1383D06
C53AD043AFFFAFFF856AA8143
91441294297FAFD501142C6BA
AA912AC3843FAFFF82856D52A
842A8692A92B8517C4451552A
816AC3844682852939 17A9542
C416928513C443A828456C3BA
91416AA92C393A82801553AAA
A81292AA814682C6A8693C6AA
A8442C6C2C1168552C16A9542
86956951692C1455416928552
C545545456C54555545444556

1,1
19,14
SWSESWSESWSSSEESEEENEESESEESSSEEESSSEEENNENEE
bash-3.2$
```

Representacion Visual:

Debemos mostra una representacion visual del laberinto ya sea en la terminal con ASCII o usando la libreria MiniLibX (MLX). La visualizacion debe mostrar claramente la entrada, salida, los muros y la solucion del laberinto. Ademas debemos agregar al menos algunas interacciones con el usuario y la representacion visual. Lo minimo indispensable seria:

- Re-generar un nuevo laberinto y mostrarlo.
- Mostrar y esconder el camino valido mas corto entre la entrada y la salida.
- Cambiar los colores de los muros.
- Opcionalmente cambiar el color del patron **42**.
- Podemos agregar mas operaciones per nee, que pereza.

Requisitos para la utilizacion del codigo:

1. Debemos implementar la generacion del laberinto como una unica clase independiente ‘MazeGenerator‘. Independiente quiere decir que no depende de nada mas para ser importada y utilizada.
2. Crear documentación en la que se describa:
    1. Como instanciar y usar el **Generator** asi como tambien un ejemplo basico.
    2. Parametros personalizados (e.g., size, seed). (No me quedo muy claro)
    3. Como acceder a la estructura generada y al menos una solucion.
3. La clase generadora del laberinto debe garantizar el acceso al laberinto pero no es obligatorio que el laberinto generado este en el mismo formato que el archivo de output, de esto se debe encargar un main.
4. El modulo entero con codigo y documentacion deben estar disponibles en un unico archivo adecuado para su instalacion posterior con pip.
5. El nombre del paquete debe ser mazegen-* y el archivo anteriormente mencionado debe estar en la raiz del repositorio. Ejemplo: mazegen-1.0.0-py3-none-any.whl .
6. Como resultado de la generacion del paquete se permiten extensiones .tar.gz y .whl.
7. En el repositorio deben ir todos los archivos necesarios para construir el paquete. En la evaluacion se instalaran de nuevo los elemetos necesarios y se recostruira el paquete desde los archivos fuente (suelere ser un setup.py).
8. El README.md debe contener una breve documentacion sobre esto (El README no es parte del modulo reusable)

Bonus:

No especifican un bonus especifico pero dan dos ejemplos:

1. Soportar múltiples algoritmos de generación de laberintos.
2. Agregar animaciones durante la generacion del laberinto.

Investigar:

- pdb: Debugger de python
- pytest y unittest: Para testing

Algoritmos:

1. Generación del laberinto: DFS. Es bueno explorando caminos en profundidad, si se encuentra con una pared, retrocede (backtracking) comienza abrirse paso de nuevo. No es bueno para encontrar para encontrar el camino mas corto, ya que su sesgo es de profundidad no de eficiencia.
2. Resolución: BFS es la mejor opción ya que antes de avanzar explora todas las capas de movimientos posibles, lo que permite elegir el camino mas corto.
3. Bonus: Algoritmo Wilson. Es muy eficiente, ya lo revisamos llegados a ese punto.

Tareas:

- [ ] Makefile
- [ ] Classe principal
- [ ] Implementacion del algoritmo
- [ ] Main
- [ ] Recepcion y verificacion de datos
- [ ] Envio de datos limpios a la clase
- [ ] Obtencion y verificacion de los datos devueltos
- [ ] Escritura del archivo output
- [ ] Implementacion de libreria visual